Causal Demand Forecasting for Rail Pricing

lucas.j.perrier@gmail.com · github.com/lucasperrier

Executive summary of a machine learning research internship at Wiremind

Lucas Perrier · Machine Learning Research Intern, Data Science R&D, Wiremind (Paris) · 22 April – 5 September 2025
Supervision: Yacine Nabet (Wiremind); weekly scientific supervision by Mathias Lécuyer (University of British Columbia)

Context and problem

Wiremind's revenue-management system (CAYZN) sets ticket prices from demand forecasts; the production forecaster is a LightGBM model. I worked on booking data from OUIGO, SNCF's high-speed rail service: each series is one scheduled train, the treatment T_t is the price on day t and the outcome Y_t is the number of seats sold that day. A forecast used for pricing is queried at prices the historical policy did not choose, so it must capture the *causal* effect of price. Observational data point the wrong way: operators raise prices when they expect high demand, and well-tuned non-causal models learn that demand rises with price (in the paper below, 13% of LightGBM and 28% of TFT test-set effects have this wrong sign).

Theoretical setting (Crasson, Nabet & Lécuyer, 2024, arXiv:2411.00126)

Prices and sales are observed up to a cut-off τ, after which prices may leave the historical distribution. The effect of T_t on Y_t is conditioned on $W_t = (S, X_{1:t}, T_{1:\tau}, Y_{1:\tau})$ (static features, calendar features, pre-cut-off history); later prices and sales follow the historical policy and would confound it. Under unconfoundedness of T_t given W_t, the conditional average treatment effect (CATE) is $\mathbb{E}[Y_t \mid T_t{=}T_2, W_t] - \mathbb{E}[Y_t \mid T_t{=}T_1, W_t]$, but a flexible regressor of Y_t on (T_t, W_t) estimates it with regularization bias on finite data. The framework instead uses the partially linear model (Robinson, 1988; Nie & Wager, 2020), extended to vector-valued treatments,

$$Y_t = T_t^\top \theta_0(W_t) + f_0(W_t) + \varepsilon_1, \quad \mathbb{E}[\varepsilon_1 \mid W_t, T_t] = 0; \qquad T_t = e_0(W_t) + \varepsilon_2, \quad \mathbb{E}[\varepsilon_2 \mid W_t] = 0.$$

The nuisances $m_0(W_t) = \mathbb{E}[Y_t \mid W_t]$ and $e_0(W_t) = \mathbb{E}[T_t \mid W_t]$ are fitted on one half of the data, and the effect model θ on the other half by minimizing the residualized loss (R-loss)

$$\ell(\theta; m, e) = \big(Y_t - m(W_t) - (T_t - e(W_t))^\top \theta(W_t)\big)^2.$$

The R-loss is Neyman-orthogonal, so nuisance error enters the bound on θ's excess risk only through its square (Foster & Syrgkanis, 2023). Price is encoded linearly (a scalar price with a W_t-dependent slope), one-hot or cumulatively over discrete price levels; the pricing forecast $\hat{Y}_t = m(W_t) + (T_t - e(W_t))^\top \theta(W_t)$ depends on T_t only through θ. Because true effects are never observed, models are scored against local effects estimated by regression discontinuity designs (RDD) at observed price changes, which rely on continuity rather than unconfoundedness. In the paper, non-causal models have the lowest in-distribution error and causal models the lowest RDD RMSE.

My contributions

- **Causal TSMixer models.** Trained and tuned the TSMixer-based nuisance (m, e) and effect (θ) models with the R-loss on OUIGO data, benchmarked them against the production LightGBM forecaster, and delivered in-distribution and causal metrics for the revision of the paper.
- **Look-ahead leakage in the non-causal baseline.** Found that the TSMixer baseline, which forecast the whole horizon from the full price sequence, could use prices of later days in its forecast for day t: its time-mixing layers span the whole sequence, and later prices are set in response to demand. Rebuilt the dataset so that each example supervises one target day t, with all later prices masked and each series replicated once per target day. The baseline then matches the paper's $f(S, X_{1:t}, T_{1:t}, Y_{1:\tau})$, and replication lowered validation loss and test error.
- **Model selection.** Dates had been assigned at random to training and validation, with the final months held out for testing, so validation measured interpolation within the training period: validation loss kept falling while test RMSE rose. A chronological train $\rightarrow$ validation $\rightarrow$ test split made the two move together, which allowed early stopping and a HyperOpt search over learning rate, dropout and weight decay.
- **Causal evaluation.** Implemented the paper's RDD protocol for these models: each model's predicted effect at a test-set price switch (its forecast at the observed price minus its forecast one price level lower) is matched to the RDD estimate at the same switch and scored by RMSE.
- **Doubly robust loss, cumulative encoding.** Implemented in Wiremind's codebase the cumulative-encoding variant of a doubly robust loss proposed in a later draft of the paper, which my supervisor then used to train models. It estimates the expected demand at the lowest price level and the incremental effect of each successive level by regressing on augmented inverse-propensity pseudo-outcomes, built from an outcome model and directly learned inverse-propensity weights; the forecast at a given price is the cumulative sum up to that level.
- **Diagnostics.** Built demand and revenue matrices by price level and days before departure, sweeping T_t through $\hat{Y}_t$ for causal models, to check that demand falls with price and that the revenue-maximizing price moves plausibly towards departure. Distribution checks caught a mis-trained m with mostly negative outputs for a non-negative demand; after retraining, m matched the demand distribution and θ was predominantly negative.

Scope

The causal-forecasting framework, the doubly robust loss, the RDD evaluation methodology and the choice of TSMixer as backbone predate the internship (Wiremind R&D and its UBC collaborator); my work concerns implementation, training, baselines, model selection, evaluation and diagnostics within that framework. Quantitative results are confidential and are not reported. Stack: YAML-configured pipelines on Kubernetes (OVHcloud) with Argo, MLflow, Docker and GitLab.